

中国科学院大学

《计算机网络(研讨课)》实验报告

姓名 寇逸欣 学号 2023K8009922004 专业 计算机科学与技术
实验项目编号 04 实验名称 交换机转发实验

一、 实验内容

1.1 广播网络,实现 broadcast_packet 函数使:

1. 网络的 host 之间可以互相 ping 通
2. 验证广播网络的效率
3. 在环状拓扑中会存在数据包环路

1.2 交换机转发,实现交换机转发逻辑使:

1. 在相同网络拓扑下,相对于 hub,链路利用效率显著提升

二、 实验内容

2.1 实验一:广播网络实验 (hub)

2.1.1 节点广播函数实现

broadcast_packet 函数实现如下:

```
// the memory of ``packet'' will be free'd in handle_packet().
void broadcast_packet(iface_info_t *iface, const char *packet, int len) // 广播数据包
{
    // TODO: broadcast packet
    // fprintf(stdout, "TODO: broadcast packet.\n");
    iface_info_t *itf = NULL;
    list_for_each_entry(itf, &instance->iface_list, list) {
        if (strcmp(itf->name, iface->name) != 0) {
            iface_send_packet(itf, packet, len);
            // log(DEBUG, "broadcast packet to %s", itf->name);
        }
    }
}
```

这个函数的作用是将收到的数据包广播到除了接收接口以外所有的接口上。

2.1.2 验证广播网络能够正常运行

即使用框架所提供的 topo 文件启动 mininet, 对于 b1 节点运行 hub 程序, 使用 ping 命令测试 h1, h2, h3 节点之间的连通性。根据以往经验推荐使用 ping dst_ip -c 2 而不是 PPT 上的 ping dst_ip -c 4, 这样更容易测出代码实现的问题。实验结果如图1所示, h1, h2, h3 节点之间均能 ping 通。

```

PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1062 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.443 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1062ms
rtt min/avg/max/mdev = 0.443/531.379/1062.316/530.936 ms, pipe 2

```

图 1: hub 广播网络 ping 通结果

由上述结果可知,广播网络能够正常运行。

2.1.3 验证广播网络的效率

下图2为 hub 广播网络拓扑结构图。在该拓扑结构中,b1 节点作为 hub 连接 h1, h2, h3 节点,h1 与 b1 之间的链路带宽为 20Mb/s,h2 与 b1 之间的链路带宽为 10Mb/s,h3 与 b1 之间的链路带宽为 10Mb/s。

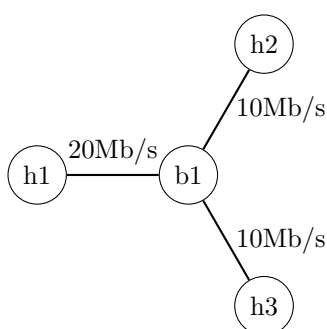


图 2: 广播网络拓扑结构

要进行 iperf 测试:

1. H1: iperf client; H2, H3: iperf servers
2. H1: iperf server; H2, H3: iperf clients

我们可以直接修改 three_nodes_bw.py 脚本, 在启动 mininet 分别在 h1, h2, h3 节点上运行 client 或 server 程序, 然后观察 iperf 的测试结果。实验结果如图3所示。

```

-----
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 44476 connected with 10.0.0.2 port 5001 (icwnd/mss/irrt=14/1448/1344666)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-15.9058 sec  5.00 MBytes  2.64 Mbits/sec
-----

Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 56720 connected with 10.0.0.3 port 5001 (icwnd/mss/irrt=14/1448/1123865)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-15.8289 sec  11.6 MBytes  6.16 Mbits/sec
-----

Client connecting to 10.0.0.1, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 42088 connected with 10.0.0.1 port 5001 (icwnd/mss/irrt=14/1448/1023432)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-15.8471 sec  14.1 MBytes  7.48 Mbits/sec
-----

Client connecting to 10.0.0.1, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.3 port 59270 connected with 10.0.0.1 port 5001 (icwnd/mss/irrt=14/1448/1031308)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-15.8871 sec  13.8 MBytes  7.26 Mbits/sec
-----

```

图 3: hub 广播网络 iperf 测试结果

从实验结果可以看出,以 h1 为 server 时,h1 到 h2 的带宽约为 7.48Mbits/sec,h1 到 h3 的带宽约为 7.26 Mbits/sec,总带宽约为 14.74Mbits/sec,带宽利用率约为 73.7%。以 h1 为 client 时,h2 到 h1 的带宽约为 2.64 Mbits/sec,h3 到 h1 的带宽约为 6.16 Mbits/sec,总带宽约为 8.8 Mbits/sec,带宽利用率约为 44%。可见,h1 为 client 时,所有数据从 h1 单一口轮流发往 h2/h3;两端返回的 TCP ACK 与数据同处一个冲突域,碰撞更频繁;且 Hub 会把发给 h2 的帧也给到 h3 链路(反之亦然),有效带宽被“重复占用”。

2.1.4 构建环形拓扑,验证该拓扑下节点广播会产生数据包环路

下图4为环形拓扑结构图。

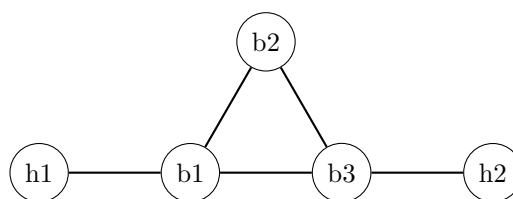


图 4: 广播网络拓扑结构

在该拓扑结构中,b1, b2, b3 节点均作为 hub 连接 h1, h2 节点,形成环形拓扑结构。在 b1, b2, b3 节点运行 hub 程序,使用 ping 命令由 h1 向 h2 发送一个数据包。通过 tcpdump 抓包,利用 wireshark 分析数据包,可以看到数据包在环形拓扑中不断循环转发,形成数据包环路,验证了环形拓扑下节点广播会产生数据包环路的现象,如图5所示。

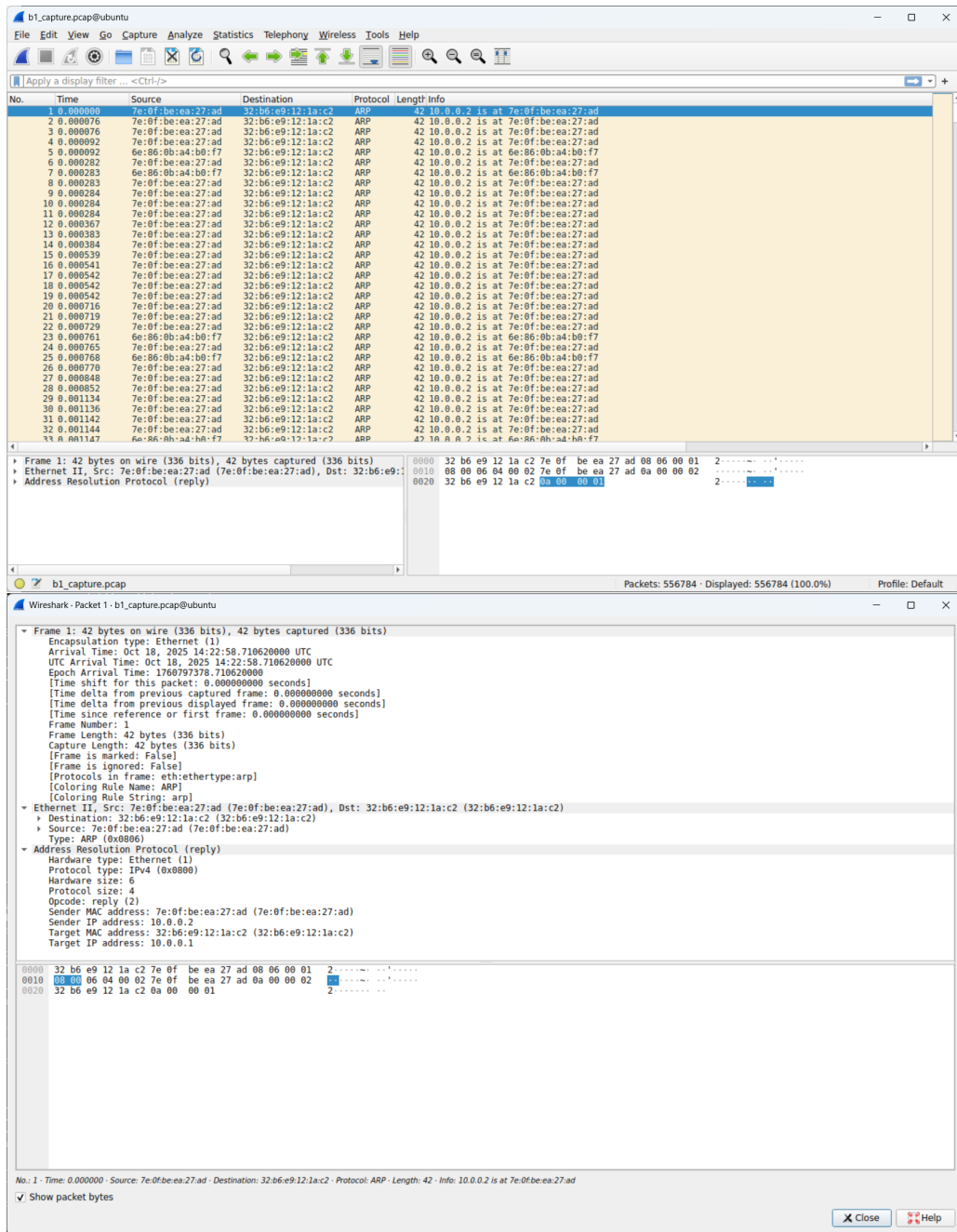


图 5: 环形拓扑数据包环路现象

2.2 实验二:交换机转发实验 (switch)

2.2.1 交换机转发逻辑实现

```
// mac.c
// lookup the mac address in mac_port table
iface_info_t *lookup_port(u8 mac[ETH_ALEN])
{
    // TODO: implement the lookup process here
    // fprintf(stdout, "TODO: implement the lookup process here.\n");
    u8 hash = hash8((char *)mac, ETH_ALEN);
```

```

pthread_mutex_lock(&mac_port_map.lock);
mac_port_entry_t *entry = NULL;
list_for_each_entry(entry, &mac_port_map.hash_table[hash], list) {
    if (memcmp(entry->mac, mac, ETH_ALEN) == 0) {
        entry->visited = time(NULL);
        pthread_mutex_unlock(&mac_port_map.lock);
        return entry->iface;
    }
}
pthread_mutex_unlock(&mac_port_map.lock);

return NULL;
}
...
// insert the mac -> iface mapping into mac_port table
void insert_mac_port(u8 mac[ETH_ALEN], iface_info_t *iface)
{
    // TODO: implement the insertion process here
    // fprintf(stdout, "TODO: implement the insertion process here.\n");
    u8 hash = hash8((char *)mac, ETH_ALEN);
    pthread_mutex_lock(&mac_port_map.lock);
    mac_port_entry_t *entry = NULL;
    list_for_each_entry(entry, &mac_port_map.hash_table[hash], list) {
        if (memcmp(entry->mac, mac, ETH_ALEN) == 0) {
            entry->iface = iface;
            entry->visited = time(NULL);
            pthread_mutex_unlock(&mac_port_map.lock);
            return;
        }
    }
}

entry = (mac_port_entry_t *)malloc(sizeof(mac_port_entry_t));
memcpy(entry->mac, mac, ETH_ALEN);
entry->iface = iface;
entry->visited = time(NULL);

list_add_head(&entry->list, &mac_port_map.hash_table[hash]);
pthread_mutex_unlock(&mac_port_map.lock);
}
...
// sweeping mac_port table, remove the entry which has not been visited in the
// last 30 seconds.
int sweep_aged_mac_port_entry()
{
    // TODO: implement the sweeping process here
    // fprintf(stdout, "TODO: implement the sweeping process here.\n");
    int cnt = 0;
    time_t now = time(NULL);
    pthread_mutex_lock(&mac_port_map.lock);

```

```

mac_port_entry_t *entry, *q;
for (int i = 0; i < HASH_8BITS; i++) {
    list_for_each_entry_safe(entry, q, &mac_port_map.hash_table[i], list) {
        if (now - entry->visited > MAC_PORT_TIMEOUT) {
            list_delete_entry(&entry->list);
            free(entry);
            cnt++;
        }
    }
}
pthread_mutex_unlock(&mac_port_map.lock);
return cnt;
}

// main.c
void handle_packet(iface_info_t *iface, char *packet, int len)
{
    // TODO: implement the packet forwarding process here
    // fprintf(stdout, "TODO: implement the packet forwarding process here.\n");

    struct ether_header *eh = (struct ether_header *)packet;
    // log(DEBUG, "the dst mac address is " ETHER_STRING ".\n", ETHER_FMT(eh->ether_dhost));

    iface_info_t *dst_iface = lookup_port(eh->ether_dhost);
    if (dst_iface) {
        iface_send_packet(dst_iface, packet, len);
    } else {
        broadcast_packet(iface, packet, len);
    }

    insert_mac_port(eh->ether_shost, iface);

    free(packet);
}

```

关键代码解释：

- `lookup_port` 函数：在 `mac_port` 表中查找目标 MAC 地址对应的接口信息。如果找到则返回对应的接口信息，并更新该条目的访问时间；如果未找到则返回 `NULL`。
- `insert_mac_port` 函数：将 MAC 地址与接口信息的映射插入到 `mac_port` 表中。如果该 MAC 地址已经存在，则更新对应的接口信息和访问时间；如果不存在，则创建一个新的条目并插入到表中。
- `sweep_aged_mac_port_entry` 函数：遍历 `mac_port` 表，删除那些在过去 30 秒内未被访问的条目，以防止表项过多导致内存浪费。
- `handle_packet` 函数：处理收到的数据包。首先查找目标 MAC 地址对应的接口信息，如果找到则将数据包发送到该接口；如果未找到则广播数据包。最后将源 MAC 地址与接收接口的信息插入到 `mac_port` 表中。

2.2.2 使用 iperf 和给定的拓扑进行实验,对比交换机转发与集线器广播的性能

使用与实验一相同的拓扑结构,只需将 hub 程序替换为 switch 程序即可。实验结果如图6所示。

```
-----
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 41754 connected with 10.0.0.2 port 5001 (icwnd/mss/irrt=14/1448/1052426)
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-16.2501 sec 18.0 MBytes 9.29 Mbits/sec
-----

Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 49622 connected with 10.0.0.3 port 5001 (icwnd/mss/irrt=14/1448/19746)
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-16.2050 sec 17.8 MBytes 9.19 Mbits/sec
-----

Client connecting to 10.0.0.1, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 46930 connected with 10.0.0.1 port 5001 (icwnd/mss/irrt=14/1448/1058951)
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-15.8621 sec 16.1 MBytes 8.53 Mbits/sec
-----

Client connecting to 10.0.0.1, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.3 port 54048 connected with 10.0.0.1 port 5001 (icwnd/mss/irrt=14/1448/1057688)
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-15.9165 sec 15.8 MBytes 8.30 Mbits/sec
-----
```

图 6: switch 交换机转发网络 iperf 测试结果

从实验结果可以看出,以 h1 为 server 时,h1 到 h2 的带宽约为 8.53Mbits/sec,h1 到 h3 的带宽约为 8.30 Mbits/sec,总带宽约为 16.83Mbits/sec,带宽利用率约为 84.15%。以 h1 为 client 时,h2 到 h1 的带宽约为 9.29 Mbits/sec,h3 到 h1 的带宽约为 9.19 Mbits/sec,总带宽约为 18.48 Mbits/sec,带宽利用率约为 92.4%。可见,交换机转发能够有效避免广播风暴和数据包环路问题,提高链路利用效率。这是因为交换机能够根据地址表进行有针对性的转发,而不是简单地广播数据包到所有接口。可以提高链路利用效率,减少冲突和碰撞的发生。

三、 实验遇到的问题

3.1 log, printf 输出导致丢包

在实验过程中,发现框架种存在大量的 DEBUG 级别的 log, printf 输出,如果这些输出输出到终端,会严重影响程序的性能,导致 ping 不通,iperf 测试结果异常的低。解决方法是将这些输出重定向到文件中,或者直接注释掉这些输出语句,这样运行过程就恢复了正常。推测是因为大量的输出操作占用了过多的 CPU 时间,导致程序无法及时处理网络数据包,从而引发丢包现象。